



POSIX - стандарт интерфейсов ОС

ISO/IEC/IEEE 9945:2009

Фактически — UNIX

Поддерживается QNX, Windows (как альтернатива)

Независимость ПО от ОС на уровне исходного текста программы

Реализация: библиотека — преобразование в системные вызовы целевой ОС

Содержание стандарта:

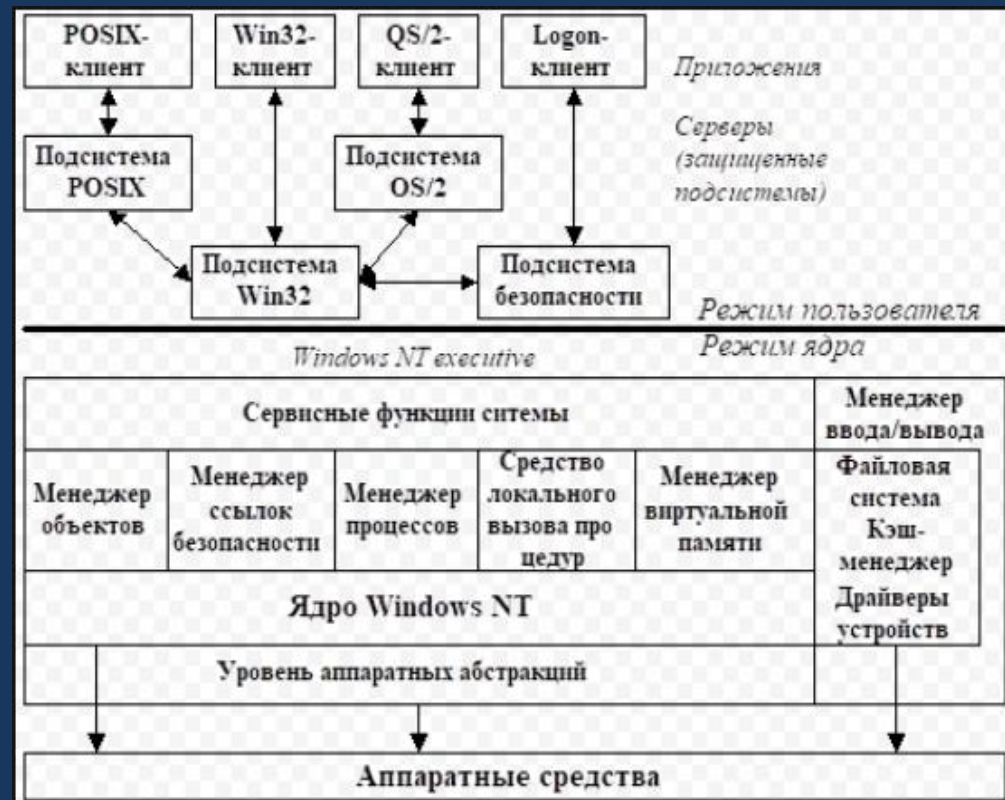
- основные определения (термины, концепции и интерфейсы, общие для всех частей);
- описание прикладного программного С-интерфейса к системным сервисам;
- описание интерфейса к системным сервисам на уровне командного языка и служебных программ;
- детальное разъяснение положений стандарта, обоснование принятых решений.



POSIX - стандарт интерфейсов ОС

В стандарте с 2003 г.:

- средства разработки
- сетевые средства
- средства реального времени
- потоки управления
- математические интерфейсы
- пакетные сервисы
- заголовочные файлы
- унаследованные интерфейсы





POSIX - стандарт интерфейсов ОС

Профили стандарта = подмножества поддерживаемых функций

PSE54 -

многопользовательская
многопроцессная, с ФС

POSIX PSE53

многопроцессная, с ФС

PSE52 однопроцессная, с
ФС

PSE51 однопроцессная,
без ФС

Сравнительная характеристика профилей

Поддерживаемые возможности	PSE51	PSE52	PSE53	PSE54
1003.1-90. Процессы	-	-	+	+
1003.1-90. Конвейеры	-	-	+	+
1003.1-90. Файлы и каталоги	-	+	-	+
1003.1-90. Пользователи и группы	-	-	-	+
1003.1b-93. Защита памяти	-	-	+	+
1003.1b-93. Часы и таймеры высокого разрешения	+	+	+	+
1003.1b-93. Сигналы реального времени	+	+	+	+
1003.1b-93. Семафоры	+	+	+	+
1003.1b-93. Разделяемая память	+	+	+	+
1003.1b-93. Межпроцессовый обмен данными	+	+	+	+
1003.1c-95. Потоки	+	+	+	+
1003.1c-95. Потокбезопасные функции	+	+	+	+
1003.1c-95. Атрибуты потоков, адрес стека	+	+	+	+
1003.1c-95. Атрибуты потоков, размер стека	+	+	+	+
1003.1c-95. Разделение процесса между потоками	-	-	+	+
1003.1c-95. Приоритеты потоков	+	+	+	+
1003.1c-95. Наследование приоритетов потоками	+	+	+	+
1003.1c-95. Защита приоритетов потоков	+	+	+	+



POSIX - стандарт интерфейсов ОС

Основные понятия:

- **Пользователь**. У пользователя есть имя и числовой идентификатор.
- **Файл** - объект, допускающий чтение и/или запись и имеющий такие атрибуты, как права доступа и тип. К числу последних относятся обычный файл, символьный и блочный специальные файлы, канал, символьная ссылка, сокет и каталог. Реализация может поддерживать и другие типы файлов.
- **Процесс** - адресное пространство вместе с выполняемыми в нем потоками управления, а также системными ресурсами, которые этим потокам требуются.
- **Терминал** (или терминальное устройство) - символьный специальный файл, подчиняющийся спецификациям общего терминального интерфейса.
- **Сеть** - совокупность взаимосвязанных хостов.
- **Языково-культурная среда** - часть пользовательского окружения, зависящая от языковых и культурных соглашений.



POSIX — работа с пользователями

Данные, ассоциированные с пользователем:

- имя пользователя
- числовой идентификатор пользователя (uid)
- числовой идентификатор начальной группы (gid)
- начальный рабочий каталог
- начальная программа пользователя
- uid=0 - суперпользователь

Данные групп:

- имя группы
- числовой идентификатор группы
- список пользователей, которым разрешено становиться членами данной группы (смена пользователем текущей группы)

```
romanow@romanow-home ~  
$ logname  
romanow  
  
romanow@romanow-home ~  
$ id  
uid=197615(romanow) gid=197121(None) группы=197121(None),114(Локальная учетная з  
апись и член группы "Администраторы"),544(Администраторы),559(Пользователи журна  
лов производительности),545(Пользователи),4(ИНТЕРАКТИВНЫЕ),66049(КОНСОЛЬНЫЙ ВХОД  
,11(Прошедшие проверку),15(Данная организация),113(Локальная учетная запись),40  
95(CurrentSession),66048(ЛОКАЛЬНЫЕ),262154(Проверка подлинности NTLM),405504(Выс  
окий обязательный уровень)
```



POSIX — работа с пользователями

```
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>
#include <stdio.h>
/* Печать элемента базы данных пользователей */
static void print_pwent (const struct passwd *pw) {
    printf ("Имя пользователя: %s\n", pw->pw_name);
    printf ("Идентификатор пользователя: %o\n", pw->pw_uid);
    printf ("Идентификатор группы: %o\n", pw->pw_gid);
    printf ("Начальный каталог: %s\n", pw->pw_dir);
    printf ("Начальная программа: %s\n", pw->pw_shell);
}
int main (void) {
    char *lgnm;          /* Имя текущего пользователя */
    struct passwd *pw;    /* Данные о текущем пользователе */
    /* Поиск и печать информации о текущем пользователе */
    if ((lgnm = getlogin ()) == NULL || (pw = getpwnam (lgnm)) == NULL) {
        fprintf (stderr, "\nНе удалось найти информацию о текущем пользователе\n");
        return 1;
    }
    printf ("\nИнформация о текущем пользователе\n");
    print_pwent (pw);
    /* То же для пользователя root */
    if ((pw = getpwuid ((uid_t) 0)) == NULL) {
        fprintf (stderr, "\nНе удалось найти информацию о пользователе root\n");
        return 1;
    }
    printf ("\nИнформация о пользователе root\n");
    print_pwent (pw);
    return 0;
}
```

Информация о текущем пользователе

Имя пользователя: romanov

Идентификатор пользователя: 601757

Идентификатор группы: 601001

Начальный каталог: /home/romanov

Начальная программа: /bin/bash

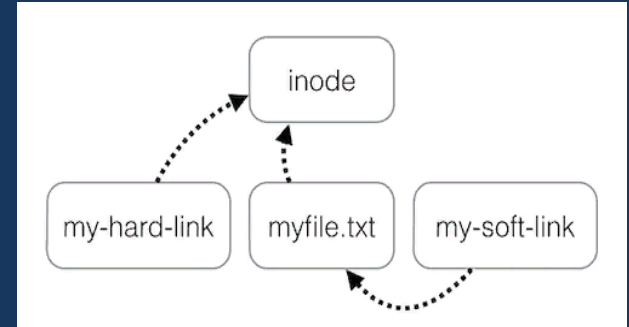
Не удалось найти информацию о пользователе root



POSIX — работа с файлами

В стандарте зафиксированы следующие типы файлов:

- обычный файл
- каталог
- канал
- символьный специальный файл
- блочный специальный файл
- сокет
- **символическая ссылка** — файл, содержащий путь к целевому файлу или каталогу (аналог ярлыка в Windows). **Жесткая ссылка** — имя файла-синоним существующего (дублирующая ссылка в каталоге на существующий inode). В отличие от ярлыка Windows символическая ссылка является частично прозрачной по отношению к функциям работы с файлом (при открывании файла-ссылки открывается целевой файл)



Определено в стандарте:

- Корневой каталог пользователя
- Символы в именах файлов (осторожно с нац. алфавитами)
- Составные(маршрутные) и простые имена, абсолютные и относительные
- Алгоритм разрешения составных имен (в т.ч. правила раскрытия символических ссылок, в т.ч. по цепочке)

```
./autofsck  
/usr/lib/tcl  
../susv3  
./14.txt
```



POSIX — работа с файлами

Характеристики файла (доступ):

- тип файла
- биты режима доступа
- числовой идентификатор владельца-пользователя
- числовой идентификатор владеющей группы

Категории доступа:

- владелец файла
- члены владеющей группы
- прочие пользователи

Для каждой категории биты доступа: чтение (r) , запись (w), исполнение (x) или поиск (каталог): `rwX - rwX — rwX`

Дополнительно: бит переустановки идентификатора пользователя, бит переустановки идентификатора группы, бит ограничения на удаление элементов каталогов



POSIX — работа с файлами

```
romanow@romanow-home ~  
$ ls -l  
итого 480  
-rwxr-xr-x  1 Unknown+User  None  62826 сен  2  2015 a.exe  
drwxr-xr-x+ 1 Unknown+User  None    0 мар  8  2015 freeglut-3.0.0  
-rwxr-xr-x  1 Администраторы None 419095 сен  3  2015 freeglut-3.0.0.tar.gz  
-rwxr-xr-x  1 Администраторы None   970 сен  2  2015 main.cpp
```

Расшифровка:

- d — каталог, b - блочный специальный файл, c - символьный специальный файл, l - символическая ссылка, p — канал, - (минус) - обычный файл
- права доступа rwxrwxrwx
- количество ссылок из каталогов
- имя владельца, имя владеющей группы
- размер
- имя



POSIX — работа с файлами

```
romanow@romanow-home ~  
$ ls -l  
итого 480  
-rwxr-xr-x  1 Unknown+User  None  62826 сен  2  2015 a.exe  
drwxr-xr-x+ 1 Unknown+User  None    0 мар  8  2015 freeglut-3.0.0  
-rwxr-xr-x  1 Администраторы None 419095 сен  3  2015 freeglut-3.0.0.tar.gz  
-rwxr-xr-x  1 Администраторы None   970 сен  2  2015 main.cpp
```

Расшифровка:

- d — каталог, b - блочный специальный файл, c - символьный специальный файл, l - символическая ссылка, p — канал, - (минус) - обычный файл
- права доступа rwxrwxrwx
- количество ссылок из каталогов
- имя владельца, имя владеющей группы
- размер
- имя



POSIX — работа с файлами

```
int main (int argc, char *argv[]) {  
    struct statvfs buf;  
    int i;
```

```
romanow@romanow-home ~
```

```
$ df
```

файловая система	1K-блоков	Использовано	Доступно	Использовано%	Смонтировано в
C:/cygwin	209720536	119645152	90075384	58%	/
D:	314576764	195354764	119222000	63%	/cygdrive/d
E:	136708920	101870948	34837972	75%	/cygdrive/e
F:	4192756	7712	4185044	1%	/cygdrive/f

```
char *cc="/cygdrive/d";
```

```
if (statvfs (cc, &buf)) {
```

```
    fprintf (stderr, "\nstatvfs: не удалось получить информацию о файловой системе, содержащей файл: %s", cc);
```

```
    return (-1);
```

```
}
```

```
printf ("\nstatvfs-информация о файловой системе, содержащей файл:\n");
```

```
printf ("Размер блока файловой системы: %ld\n", buf.f_bsize);
```

```
printf ("Базовый размер блока файловой системы: %ld\n", buf.f_frsize);
```

```
printf ("Общее число блоков базового размера в файловой системе: %ld\n", buf.f_blocks);
```

```
printf ("Общее число свободных блоков: %ld\n", buf.f_bfree);
```

```
Общее число свободных блоков: 29805498
```

```
Число свободных блоков, доступных непривилегированным процессам: 29805498
```

```
Общее число описателей файлов: -1
```

```
Общее число свободных описателей файлов: -1
```

```
Число описателей файлов, доступных непривилегированным процессам: -1
```

```
Идентификатор файловой системы: 900590178
```

```
Битная шкала флагов: 3e700ff
```

```
Максимальная длина имени файла: 255
```



POSIX — работа с файлами

```
#include <fcntl.h>

int creat (const char *path, mode_t mode);
```

```
#include <sys/stat.h>

int mkdir (const char *path, mode_t mode);
```

```
#include <unistd.h>

int link (const char *source_path,
         const char *target_path);

#include <unistd.h>

int symlink (const char *link_contents,
            const char *link_name);
```

Создание символьных и жестких ссылок

```
romanow@romanow-home ~
$ ln main.cpp main1.cpp
ln: не удалось создать жёсткую ссылку «main1.cpp» => «main.cpp»: Permission denied

romanow@romanow-home ~
$ ln -s main.cpp main1.cpp

romanow@romanow-home ~
$ ls
a.exe  freeglut-3.0.0  freeglut-3.0.0.tar.gz  main.cpp  main1.cpp
```

File Edit Options Encoding Help

```
00000000: 21 3C 73 79 6D 6C 69 6E|6B 3E FF FE 6D 00 61 00 | ?<symlink>яяя.а.
00000010: 69 00 6E 00 2E 00 63 00|70 00 70 00 00 00 00 00 | i.n...c.p.p...
```



POSIX — работа с файлами

```
#include <unistd.h>
int unlink (const char *path);
#include <unistd.h>
int rmdir (const char *path);
#include <stdio.h>
int remove (const char *path);
```

```
#include <stdio.h>
int rename (const char *old_path,
            const char *new_path);
```

Символическая ссылка как файл читается отдельной функцией (содержимое файла в буфер), иначе через нее открывается указуемый файл

```
#include <unistd.h>
ssize_t readlink (const char *restrict
                  link_name, char *restrict buf,
                  size_t buf_size);
```



Текстовые и двоичные файлы (cprog)

Проблемы текстовых потоков, не учитываемые в POSIX:

- не делает разницы между текстовым и двоичным содержимым
- проблема кодирования «конца строки» в разных ОС — Windows требует перекодировки
- не решается проблема кодовых таблиц

Все файлы являются двоичными.

Разница между "текстовыми файлами" и "двоичными файлами" в контексте `fopen` заключается в том, что стандартная библиотека может использовать фильтр для чтения и записи файла в текстовом режиме. В двоичном режиме всегда записывается то, что вы пишете.

Единственный практический пример, который я знаю, - это в Windows, где открытие файла в текстовом режиме заставит его конвертировать между окончаниями строк в стиле Windows (CRLF, `"\r\n"`) и концами строк в стиле C/style (Unix) LF, `"\n"`). Например, когда вы открываете файл для чтения в Windows в текстовом режиме, окончание строк будет отображаться как `"\n"` в вашей программе. Если вы откроете его в двоичном режиме, окончание строк будет выглядеть как `"\r\n"` в вашей программе.

Windows:

- Открытие файлов — `r`, `rt`, `rb` (text/binary)
- Преобразование символа «конца строки» ????



Символы. Строки. Текст

Медицинские факты:

Представление символов и текста базируется на форматах внутреннего представления данных в компьютере - **машинное слово, целое со знаком и без знака**

Символу соответствует машинное слово - байт (char), 2 байта (short) или переменной длины (1-2) в зависимости от способа кодирования = **внутреннее представление символа** или **код символа**

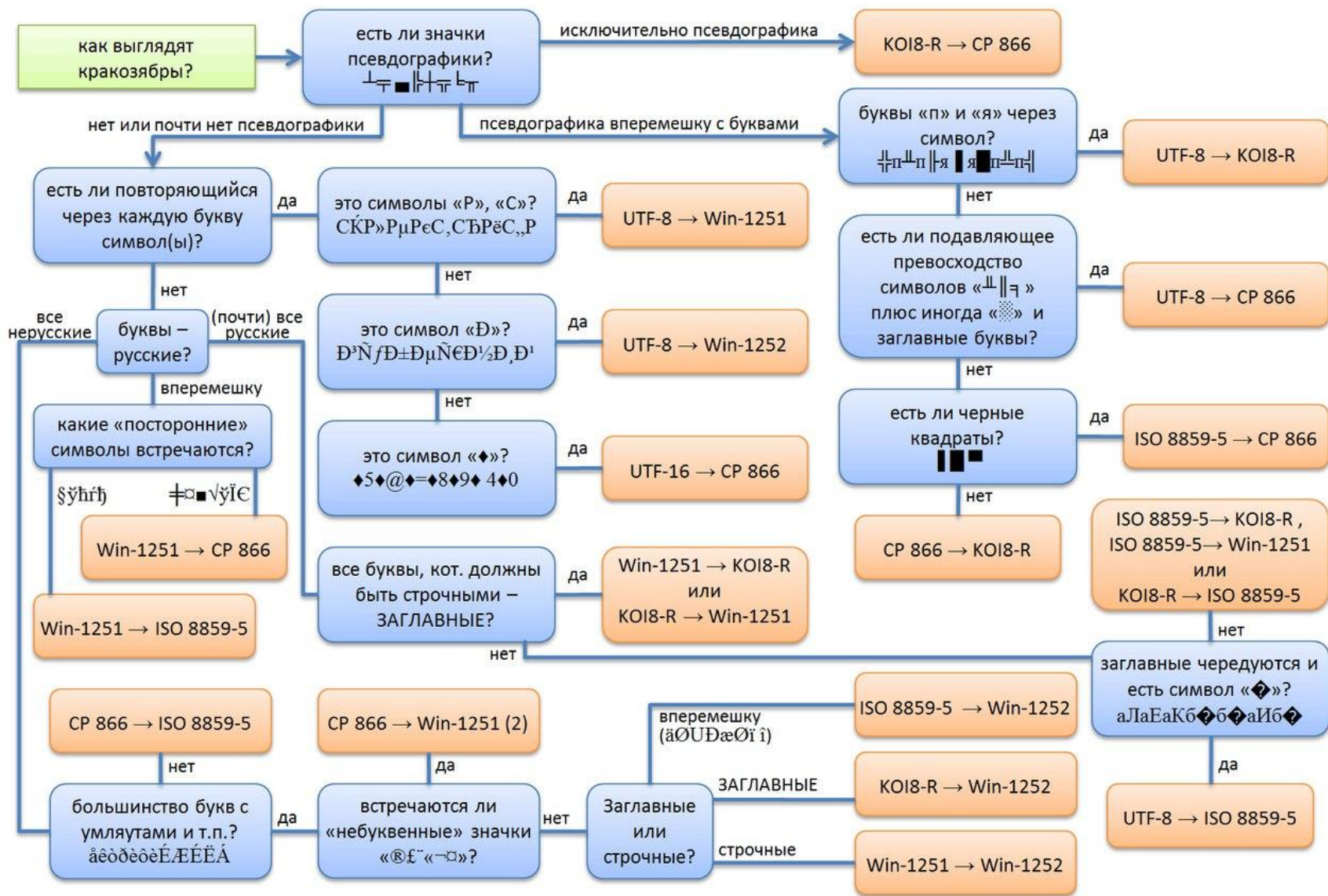
Кодировка – правило соответствия символа его коду - **кодовая таблица**
в Си, Java символы напрямую отображаются на базовые ТД, в Бейсик, Паскаль – функции псевдо-преобразования символа в код

Анахронизмы – многообразие, сложности и нестыковки форматов представления исходят из исторических реалий представления текста и файлов

в файле не содержатся данные о его кодировке
тип (расширение) файла формально не связаны с его содержимым (на уровне **соглашения** о типах)

например, в html-файле есть тег о смене кодировки **<meta http-equiv="Content-Type" content="text/html; charset=windows-1251">**

в Си кодовая таблица назначается библиотекой





Кодировка символов

Байтная – 1 символ = 1 байт, 60-70 гг., 256 символов, кодовая таблица, классический Си, тип данных - char:

первые 128 символов для все кодовых таблиц совпадают (латиница, цифры)
кодвые таблицы для кириллицы:

- кодвая таблица Windows **CP-1251**

- кодвая таблица DOS **CP-866**;

- кодвая таблица Международной организации стандартизации (**ISO**), используемая семействами мобильных ОС UNIX, Linux, FreeBSD и т.п. - **ISO-8859-5**;

- кодвые таблицы «советских» стандартов кодов информационного обмена (**KOI-8**) - **CP KOI-8U** и **CP KOI-8R**

консольное приложение в Windows в VisualStudio в DOS-кодировке,
обработка – в Windows (CP-1251), для кириллицы требуется перекодировка,
при работе в NetBeans – не требуется



Кодировка символов (байтная)

Таблица 2. Кодовая страница CP866

	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
8.	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
9.	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
A.	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
B.	q	w	e	r	t	y	u	i	o	p	~	^	^	^	^	^
C.	l	l	l	l	l	l	l	l	l	l	l	l	l	l	l	l
D.	l	l	l	l	l	l	l	l	l	l	l	l	l	l	l	l
E.	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
F.	Е	ё	Є	є	І	і	Ў	ў	°	·	·	√	№	□	■	А0

DOS CP-866;

Windows CP-1251

«советские» стандарты
(KOI-8) - CP KOI-8U и CP KOI-8R

Таблица 3. Кодовая страница CP1251

	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
8.	Ђ	Ѓ	,	Ѕ	„	…	†	‡	€	‰	Љ	‹	Њ	Ќ	Ќ	Ц
9.	ђ	ѓ	,	ѕ	„	…	—	—		™	љ	›	њ	ќ	ћ	ц
A.		Ў	ў	Ј	Љ	Ѓ	Ѕ	€	©	€	”	—		®	Ї	
B.	°	±	І	і	г	μ	¶	·	ё	№	є	”	ј	Ѕ	ѕ	ї
C.	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
D.	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
E.	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
F.	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я

Таблица 4. KOI8-R

	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
8.	Љ	Ј	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
9.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
A.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
B.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
C.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
D.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
E.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ
F.	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ	Ѓ	Ѕ

8.	п	я	р	с	т	у	ж	в	ь	ы	з	ш	э	щ	ч	ъ
9.	ю	а	б	ц	д	е	ф	г	х	и	й	к	л	м	н	о
F.	п	я	р	с	т	у	ж	в	ь	ы	з	ш	э	щ	ч	ъ



 D:\Temp\cpp1VS\cpp1\Debug\cpp1.exe

```
abcd
querty
string1
string two
12345|XXXXXXXXXXXXXXXX|string1
string two
qqqqq|XXXXXXXX|Tb+oPn>
abcd
Введите строку с прописными буквами
вапр
ВАПР
Значение=66
Введите строку с прописными буквами
вапры
ВАПРЫ
```

Архив
cprog/programms/russian.cpp



Кодировка символов (Unicode-1,2)

UNICODE – универсальная система кодирования символов (коды как двоичные числа, имеется несколько вариантов преобразования во внутреннее представление и последовательные потоки). Кодовое пространство UNICODE расширяется

Обозначение U+1D788 – код в 16-ой СС

UNICODE-1 (1991) – 1 символ = 16 разрядов (4 цифры 16СС), 65536 символов. Первые 128 символов совпадают с байтной (латиница, цифры)

UNICODE-2 (1996) - 1 символ = 4 байта, в т.ч. иероглифы, эмодзи

0x0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
400		Ё	Ъ	Ѓ	Є	Ѕ	І	Ї	Ј	Љ	Њ	Ћ	Ќ		Ў	Џ
410	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
420	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
430	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
440	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
450		ё	ђ	ѓ	є	ѕ	і	ї	ј	љ	њ	ћ	ќ		ў	џ
490	Ѓ	ѓ	ƒ	ѓ			Ж	ж			Қ	қ				
4A0			Ң	ң											Ү	ү
4B0	Ң	ү	Ҳ	ҳ							Һ	һ				
4D0									Ә	ә						
4E0									Ө	ө						



Кодировка символов (UTF-8,16,32)

UTF (UNICODE Transformation Format) - кодировка Unicode в формат последовательной передачи в потоке.

UTF8 - кодировка Unicode последовательностью переменной длины

0x00000000 — 0x0000007F: 0xxxxxxx — первые 128 -> 1 байт

0x00000080 — 0x000007FF: 110xxxxx 10xxxxxx — 2 байта -> $2^{11} = 2048$

0x00000800 — 0x0000FFFF: 1110xxxx 10xxxxxx 10xxxxxx

0x00010000 — 0x001FFFFFF: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Символ	Двоичный код символа	UTF-8 в двоичном виде	UTF-8 в шестнадцатеричном виде
\$ U+0024	0100100	00100100	24
¢ U+00A2	10100010	11000010 10100010	C2 A2
€ U+20AC	100000 10101100	11100010 10000010 10101100	E2 82 AC
⌘ U+10348	1 00000011 01001000	11110000 10010000 10001101 10001000	F0 90 8D 88

UTF16 - двухбайтные слова (short), область 0xD800—0xDFFF первого слова, отведена для «суррогатных пар»

- Code $\leq \text{FFFF}_{16}$ записываются 16-битным словом (исключая диапазон для суррогатов).
- Code $10000_{16}..10\text{FFFF}_{16}$ (больше 16 бит)
 - Code - $10000_{16} = 0...\text{FFFF}_{16}$, (20 бит)
- Старшие 10 бит + D800_{16} , = первое слово $\text{D800}_{16}..\text{DBFF}_{16}$.
- Младшие 10 бит + DC00_{16} , = второе $\text{DC00}_{16}..\text{DFFF}_{16}$.



Кодировка символов Unicode

UTF-32 – 1 символ = 4 байта (32 разряда) – прямое представление UNICODE

BOM – Byte Order Mark – идентификатор формата

UTF-8 EF BB BF

UTF-16 BE FE FF

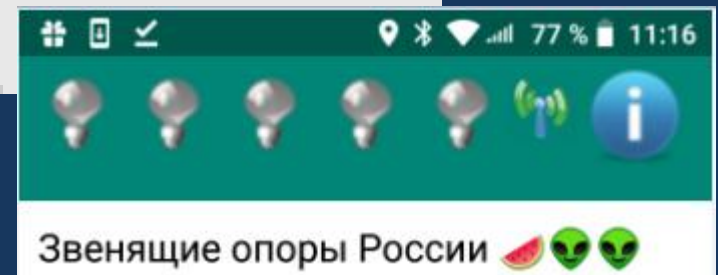
UTF-16 LE FF FE

UTF-32 BE 00 00 FE FF

UTF-32 LE FF FE 00 00

Java: char = short – 2 байта. Кодировка UTF-16

```
//-----  
int[] surrogates = {0xD83D, 0xDC7D};  
String title = "Звонящие опоры России "+  
    new String(Character.toChars( codePoint: 0x1F349 ))+  
    new String(surrogates,  offset: 0, surrogates.length)+  
    "\uD83D\uDC7D";  
addToLog(title, textSize: 20);  
}
```





Кодировка символов

Пример перекодировки кириллицы из UTF8 в CP1251 – диапазон кириллицы из двухбайтного представления склеивается в байт и переносится в диапазон начала кириллицы в CP1251

```
void UTFtoChar() {
    char *s=strdup(str);
    int i,j;
    for(i=0,j=0; s[i]!=0;j++) {
        if((s[i] & 0x0E0) != 0x0C0) //0xC0 = [110]0 0000
            str[j]=s[i++];        //0xE0 = маска [111]0 0000
        else {                    //три старших разряда = 110
            int v=s[i++] & 0x03;  //2 разряда из первого и 6 из второго
            v = (v<<6 | s[i++] & 0x03F) - 0x10 + 0xC0;
            str[j]=v;              // Начало кириллицы в CP1251 = 0xC0
        }                        // Начало кириллицы в UNICODE = 0x410 (0x10)
    }
    delete []s;
    str[j]=0;
}
```

```
//-----
int[] surrogates = {0xD83D, 0xDC7D};
String title = "Звенящие опоры России "+
    new String(Character.toChars( codePoint: 0x1F349))+
    new String(surrogates, offset: 0, surrogates.length)+
    "\uD83D\uDC7D";
addToLog(title, textSize: 20);
}
```

Звенящие опоры России 🍉👽👽



Строка

Строка – последовательность символов

Строка –
последовательность
символов в памяти

Символ перехода к
следующей строке
(конца строки)- **ln**

Си: последовательность,
ограниченная символом с кодом
0 (\0) – конец строки в памяти

Тип данных – строка: целое
(short) - счетчик символов
(байтов) + последовательность
байтов
Паскаль, Бейсик, Java-вывод в
двоичный поток writeUTF

0	5	'С'	'т'	'р'	'о'	'к'	'а'
---	---	-----	-----	-----	-----	-----	-----



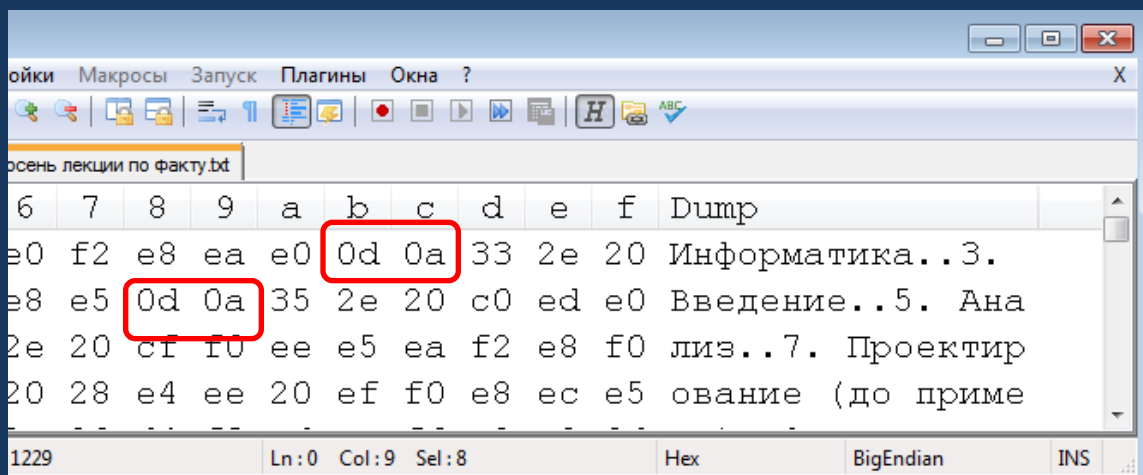
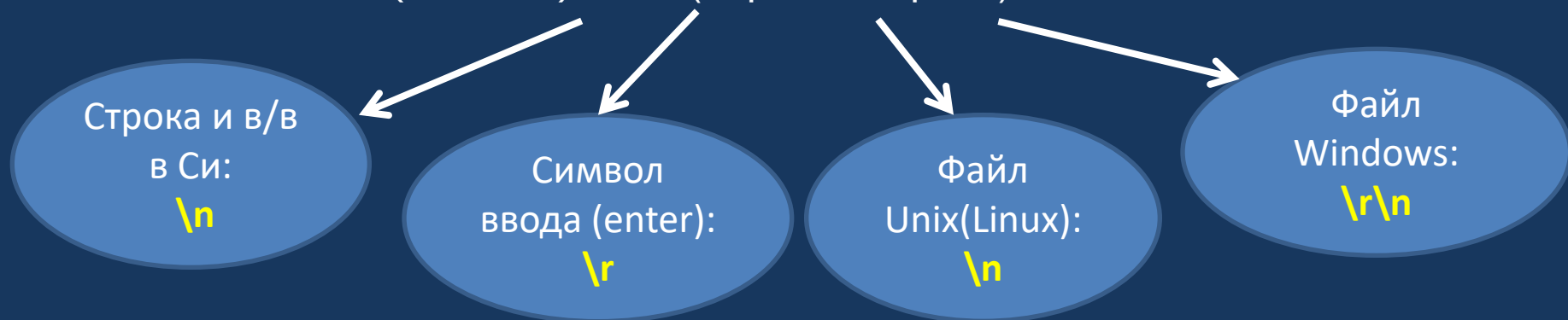
Строка

Символ перехода к следующей строке (конца строки)

Используемые коды символов:

0x0D – CR (carriage return) – ВК (возврат каретки) - **\r**

0x0A – LF (line feed) – ПС (перевод строки) - **\n**





Текстовые и двоичные файлы (сprog)

Модель двоичного файла: «расширяющийся» массив физических байтов

```
int fread (void *buf, int size, int nrec, FILE *fd);
int fwrite (void *buf, int size, int nrec, FILE *fd);
```

```
// Прочитать целую переменную и следующий за ней
// динамический массив из n переменных типа double
int n; // в целой переменной – размерность массива
fread(&n, sizeof(int), 1, fd); // указатель на переменную int
double *pd = new double[n];
fread(pd, sizeof(double), n, fd); // преобразование к void* - неявное
```

Последовательный доступ — по умолчанию,

произвольный — позиционирование

int fseek(FILE *fp, long pos, int mode) — установить позицию

long ftell(FILE *fp) — читать абсолютную позицию

#define SEEK_SET 0 Относительно начала файла, позиция 0

#define SEEK_CUR 1 Относительно текущей позиции,
>0 - вперед, <0 - назад

#define SEEK_END 2 Относительно конца файла, pos<0, pos=0 - EOF



Текстовые и двоичные файлы (сprog)

```
long fsize;  
fseek(fl, 0L, SEEK_END); // Установить позицию на конец файла  
fsize = ftell(fd);       // Прочитать значение текущей позиции
```

```
fseek(fd, 100, SEEK_SET); // По адресу 100 находится смещение  
fread(&P, sizeof(long), 1, fd); // Читается P=46, после чтения текущая позиция  
fseek(fd, i, SEEK_CUR); // 100+sizeof(long)=104, позиционирование 104+46=150
```

```
// Открыть существующий как двоичный для чтения и записи  
FILE *fd; fd = fopen("a.dat", "rb+wb");  
// Создать новый как двоичный для записи и чтения  
fd = fopen("a.dat", "wb+");
```





Текстовые и двоичные файлы (сprog)

```
//-----94-00.cpp
// Формирование ДАМПА для чтения файла
void main(){
FILE *fd=fopen("94-00.dat","wb");
char k=10,m=4;
short A[10]={6,3,7,3,4,8,300,5,23,64};
int B[4]={6,3,7,3};
long p=0,offset;
fwrite(&p,sizeof(long),1,fd);      // Занять место под указатель
fwrite(&k,1,1,fd);                 // Записать один байт - счетчик
fwrite(A,sizeof(short),k,fd);     // Записать массив коротких целых (2B)
p=ftell(fd);                      // Получить значение указателя
fwrite(&m,1,1,fd);                 // Записать один байт - счетчик
fwrite(B,sizeof(int),m,fd);       // Записать массив целых
fseek(fd,0,SEEK_SET);             // К началу файла
fwrite(&p,sizeof(long),1,fd);     // Обновить указатель на второй массив
fclose(fd);}

```

адрес строки

адрес массива 00000019₁₆

char 0A₁₆=10₁₀

Листер - [E:\romanow\Book2008\Programms\94-00.dat]

Файл Правка Вид Справка 100 %

00000000: 19 00 00 00 0A 06 00 03 00 07 00 03 00 04 00 08 |

00000010: 00 2C 01 05 00 17 00 40 00 04 06 00 00 00 03 00 |

00000020: 00 00 07 00 00 00 03 00 00 00 00 00 00 00 00 00 |

0 1 2 3 4 5 6 7 8 9 A B C D E F

младшая цифра адреса

short 012C₁₆=300₁₀

адрес 19₁₆

int 00000006₁₆=6₁₀